# University Management System
# Mutation Testing Report

CSE 731: Software Testing

Term I (2025–26)


**Team Members:**

Hemant Gupta — IMT2022030

Hemang Seth — IMT2022098

International Institute of Information Technology Bangalore


November 26, 2025

**Abstract**

This report documents the design, implementation, and extensive mutation testing of a University Management System developed as part of the CSE 731 Software Testing course at IIIT Bangalore. The system models multiple administrative and academic functions including student enrollment, faculty management, library operations, and departmental budgeting. It incorporates rich logical interactions, strong validation rules, stateful components, and computational behaviors that provide a robust foundation for rigorous testing.

The test suite accompanying the project was crafted not merely for functional assurance but to demonstrate the advanced applicability of mutation testing using PITest. With an achieved line coverage of 98 % and a line coverage of 98 % , mutation coverage 80 % , test strength 83 % across 222 mutants, the project demonstrates the depth, precision, and effectiveness of a well-engineered test suite.

# Contents


I Introduction . . . . . 1
II Project Description . . . . . 2
III Folder Structure and Components . . . . . 3
IV Testing Methodology . . . . . 4
V Unit Testing . . . . . 5
VI Integration Testing . . . . . 5
VII Test Case Design . . . . . 5
VIII Mutation Testing . . . . . 8
IX Execution and Tooling . . . . . 10
X Code Flow Summary . . . . . 11
X.1 Setup and Initialization . . . . . 11
X.2 Enrollment and Integration Logic . . . . . 11
X.3 Academic State Update . . . . . 12
X.4 Library Workflow and Utility Operations . . . . . 12
X.5 Test Verification Flow . . . . . 12
XI AI Usage Declaration . . . . . 13
XII Conclusion . . . . . 13


# I Introduction

Software testing is a foundational discipline in software engineering, essential for validating reliability, coherence, and correctness in complex systems. Traditional metrics such as line coverage and branch coverage are limited in their ability to measure the strength of a test suite. A line may be executed, yet the assertions may fail to detect a subtle logical fault. Mutation testing directly addresses this limitation by inserting small artificial modifications, known as mutants, and measuring how many of these modifications are detected and rejected by the test suite.

The University Management System developed in this project provides a realistic and multi-layered environment for demonstrating mutation testing. It is constructed with numerous interacting components—each containing boundary-sensitive logic, state

accumulation, arithmetic operations, and decision-making rules. These attributes make the system ideal for mutation testing research and educational demonstration.

# II Project Description

The University Management System is a comprehensive backend software model that simulates real-world academic and administrative activities of a university. The system spans several critical functions:

- Student enrollment, GPA calculation, credit tracking, and eligibility evaluation.
- Faculty records such as tenure determination, salary management, and years of service.
- Course management, capacity enforcement, and cancellation.
- Departmental budgeting and hiring constraints.
- Library operations including borrowing, returning, book damage tracking, and overdue fine processing.

The project was intentionally designed to include boundary conditions, multi-class interactions, accumulation logic, and strict validation processes. These characteristics produce an excellent mutation testing landscape, as numerous mutants become meaningful and difficult to kill unless the test suite is strong.

# Project Goal

As defined by the course guidelines for CSE 731, the project was developed to "stand out in terms of demonstrating the applicability of a particular technique and one or more tools." Mutation testing, a sophisticated and industry-relevant evaluation technique, was chosen as the central theme. The system was engineered not only for correctness but also to showcase the role of mutation testing as a robust measure of test adequacy.

# Why This Project Stands Out

The project distinguishes itself for several reasons:

First, the test suite goes beyond traditional line coverage achievements. Although the project attains 98 % line coverage, the more important metric is the line coverage of 98 % , mutation coverage 80 % , test strength 83 %, which indicates an ability to catch subtle logic faults.

```
UniversityMutationProject/
├── images/
│   └── mutation_report.png    # Mutation test report visualization
├── pom.xml                    # Maven configuration with PITest plugin
├── README.md                  # Project documentation
└── src/
├── main/java/com/university/
│   ├── Main.java              # Entry point
│   ├── Person.java            # Abstract base class
│   ├── Student.java           # GPA, credits, probation logic
│   ├── Faculty.java           # Tenure, salary, service years
│   ├── Course.java            # Capacity, enrollment, cancellation
│   ├── Department.java        # Budget calculation, hiring logic
│   ├── LibraryBook.java       # Book state (damage, availability)
│   ├── LibrarySystem.java     # Overdue fine calculation
│   ├── EnrollmentService.java # Integration of Student-Course rules
│
└── test/java/com/university/
    ├── StudentTest.java           # Student GPA, credits, probation tests
    ├── FacultyTest.java           # Tenure, salary, service year tests
    ├── CourseTest.java            # Capacity, enrollment, cancellation tests
    ├── DepartmentTest.java        # Budget calculation, hiring logic tests
    ├── LibraryBookTest.java       # Book state (damage, availability) tests
    ├── LibrarySystemTest.java     # Overdue fine calculation tests
    ├── EnrollmentServiceTest.java# Integration of Student-Course rules tests
    ├── MainTest.java              # System entry point tests
    ├── PersonTest.java            # Abstract base class tests
    ├── ExceptionBoosterTest.java # Exception handling tests
    ├── BoundarySniperTest.java    # Conditionals boundary tests
    ├── SniperRound2Test.java      # Math and boolean logic tests
    ├── StateAccumulationTest.java# Accumulation of fine tests
    └── OutputCaptureTest.java     # System.out and System.err interception tests
```

Figure 1: Folder structure of the project

Second, unlike simple projects where business logic is concentrated in a single monolithic class, this system deliberately enforces separation of concerns. Entities such as `Student`, `Course`, and `Faculty` function as data models while business rules are encoded in service-oriented classes such as `EnrollmentService`. This architectural approach avoids the creation of "God objects" and provides a rich structure for mutation analysis.

Third, the project demonstrates advanced tooling. Tests such as `OutputCaptureTest` intercept `System.out` and `System.err` streams to detect mutants arising from void method call removal—an advanced use-case of mutation testing seldom demonstrated in standard coursework.

# III Folder Structure and Components

The entire system consists of roughly one thousand lines of code distributed across multiple well-organized packages. The figure explains the folder structure and the table below tells about the project

| **Folder / File** | **Description** |
|---|---|
| UniversityMutationProject/ | Project root |
| images/mutation_report.png | Mutation testing report visualization |
| pom.xml | Maven configuration including PITest integration |
| README.md | Project documentation |
| Main.java | Entry point for the full system execution |
| Person.java | Abstract base class providing identity and validation logic |

| | |
|---|---|
| Student.java | GPA calculation, credit management, probation status |
| Faculty.java | Tenure evaluation, salary logic, service year accumulation |
| Course.java | Enrollment capacity, cancellation, and seat management |
| Department.java | Faculty list, salary budget logic, and hiring constraints |
| LibraryBook.java | Damage tracking, usability determination, borrowed state |
| LibrarySystem.java | Borrow/return logic, fine calculation, including year wrap-around |
| EnrollmentService.java | Integrates all enrollment rules and cross-module interactions |
| StudentTest.java | Tests related to GPA, credit, and probation rules |
| FacultyTest.java | Salary raise and tenure boundary testing |
| CourseTest.java | Enrollment limits, cancellation logic |
| DepartmentTest.java | Budget constraints and salary aggregation testing |
| LibraryBookTest.java | Damage, usability, and borrow flow testing |
| LibrarySystemTest.java | End-to-end borrowing and fine calculation |
| EnrollmentServiceTest.java | Probation, capacity, and credit limit enforcement |
| MainTest.java | Ensures system entry point executes without failure |
| PersonTest.java | Validation of base class logic including email and age boundaries |
| ExceptionBoosterTest.java | Tests designed to kill statement deletion mutants |
| BoundarySniperTest.java | Boundary-specific mutation targets |
| SniperRound2Test.java | Negation and arithmetic mutation killers |
| StateAccumulationTest.java | Accumulation-focused mutation killers |
| OutputCaptureTest.java | Detects mutants that remove print/logging statements |

# IV Testing Methodology

The testing strategy follows industry best practices and academic guidelines, built explicitly to handle mutation testing challenges. It is divided into unit testing and integration testing.

# V Unit Testing

These tests ensure each component behaves correctly in isolation. The following table describes the unit test classes and their objectives:

| **File** | **Focus** | **Primary Goal** |
|---|---|---|
| StudentTest | Student and Person logic | Validate GPA calculation, credit accumulation, and probation logic |
| FacultyTest | Faculty logic | Confirm salary updates and tenure boundaries |
| LibraryBookTest | Book state | Confirm damage accumulation and usability determination |

# VI Integration Testing

Integration tests ensure cohesion among interacting modules.

| **File** | **Focus** | **Primary Goal** |
|---|---|---|
| EnrollmentServiceTest | Student + Course | Enforces eligibility, probation rules, and capacity limits |
| DepartmentTest | Faculty + Budget | Validates salary aggregation and hiring limits |
| LibrarySystemTest | Student + Library-Book | Validates borrow/return cycle and fine calculation |

# VII Test Case Design

The test suite was carefully constructed to target mutation–sensitive behaviors across the entire system. Rather than relying on generic correctness tests, the suite explicitly adopts classical software testing design techniques such as Boundary Value Analysis (BVA), Equivalence Partitioning (EP), State Accumulation Testing, Logic Negation Testing, and Exception Enforcement. Each of these strategies was selected because mutation testing operators tend to introduce faults precisely in these areas. The following subsections elaborate on how each technique was applied and which classes of mutants were successfully eliminated as a result.

## Boundary Value Analysis (BVA)

Many critical rules in the system rely on strict numerical and logical thresholds, making them highly sensitive to boundary–altering mutants (e.g., replacing $>$ with $\geq$, $<$ with $\leq$, or shifting constants). BVA was applied extensively in tests such as:

- Tenure eligibility in `Faculty`: testing exactly 5 years (not eligible) and 6 years (eligible) to kill the Conditional Boundary Mutator that changes $>$ to $\geq$.
- Book usability in `LibraryBook`: testing damage levels 79 and 80 to detect mutants that alter the comparison from $damage < 80$ to $damage \leq 80$.
- Academic probation in `Student`: testing GPA exactly at 2.0 to catch mutants negating or widening the probation threshold.
- Credit limit enforcement in `EnrollmentService`: testing enrollment at exactly 20 credits (allowed) and 21 (disallowed) to kill $>/\geq$ mutants.

These tests successfully eliminated mutants altering boundary conditions, off-by-one logic, and fencepost errors.

## Equivalence Partitioning

Although the system is dominated by numeric boundaries, several logical structures were tested using equivalence classes:

- Valid vs. invalid email formats in `Person`.
- Valid vs. invalid credit inputs in `Student`.
- Valid vs. invalid salary inputs in `Faculty`.
- Valid vs. invalid course capacity values in `Course`.

These partitions helped ensure that input validation mutants (such as Statement Deletion Mutators that remove guard clauses) could not survive.

## State Accumulation Testing

A significant number of mutation operators affect arithmetic behavior. To eliminate these, the suite included several tests that rely on repeated state updates:

- Calling `Student.updateAcademicRecord` twice ensures that the $+=$ operator is preserved, killing Assignment Replacement Mutators that change accumulation to overwriting.

- Calling `Faculty.incrementServiceYears` thrice verifies correct year accumulation, eliminating mutants that reset or replace the accumulator.
- Repeated fine accumulation in `LibrarySystem` ensures that arithmetic mutants in the overdue fine calculation are killed.
- Two-faculty budget accumulation in `Department` ensures that total salary is a sum, eliminating mutants that change $total+ = salary$ to $total = salary$.

This class of mutations is notoriously hard to kill without deliberate accumulation-based tests.

## Logic Negation Testing

Mutation testing frequently negates conditions or removes parts of Boolean expressions. To address this:

- Duplicate enrollment checks in `Course` (`!list.contains(student)`) ensure that removing the negation or the duplicate check fails the test.
- Email validation in `Person` requires both "@" and "."; tests explicitly evaluate formats missing one or both components, killing mutants that remove half of the condition.
- Probation logic in `Student` relies on two conditions; mutants deleting one clause are reliably detected.

These tests killed Conditional Negation Mutators, Conjunction/Disjunction Mutators, and Partial Condition Removal Mutators.

## Exception Enforcement Tests

The system includes numerous validation conditions that throw exceptions when provided invalid input. Statement Deletion Mutators often delete these validation statements. To eliminate such mutants:

- `ExceptionBoosterTest` enforces exceptions for invalid grades, age boundaries, negative salaries, invalid course creation arguments, and more.
- Constructor tests across `Student`, `Faculty`, `Course`, and `Department` detect modifications to guard clauses.

These tests ensure that no mutation altering defensive programming behaviors can survive.

### Void Method Call Mutation Detection

An advanced feature of this project involves killing Void Method Call Mutators, especially those that silently remove `System.out.println` or `System.err.println` calls.

`OutputCaptureTest` redirects both stdout and stderr, compares actual output with expected substrings, and ensures that mutants removing print statements do not survive. This is a unique and advanced mutation-killing technique.

In summary, the test suite was intentionally engineered, rather than passively written, to attack all major classes of mutation operators. It systematically kills boundary mutants, arithmetic mutants, Boolean negation mutants, condition removal mutants, exception removal mutants, and void-call deletion mutants. This strategic design explains the strong line coverage of 98 % , mutation coverage 80 % , test strength 83 % despite a complex codebase with thousands of execution paths.

## VIII Mutation Testing

| **Tool Feature Demonstrated** | **Project Application** | **Significance** |
|---|---|---|
| Mutation Coverage Reporting | Generated a formal HTML report that isolates survived mutants for analysis. | Shows the tool can objectively measure test-suite effectiveness beyond line coverage. |
| JUnit 5 Integration | Configured the pitest-junit5-plugin in `pom.xml` to connect PIT with JUnit 5. | Ensures mutation tests run reliably on a modern test framework used in industry. |
| Output Stream Interception | Created `OutputCaptureTest.java` to verify that print/log statements were executed. | Kills Void Method Call Mutators by detecting removed `println` calls—an advanced mutation scenario. |

Table 4: Comprehensive Tool Demonstration Using PITest

Using PITest, a complete mutation analysis was conducted. The results are as follows:

- Line Coverage: 98 %
- Mutations Generated: 222
- Mutations Killed: 178

- Surviving Mutants: 44
- Mutation Coverage: 80 %
- Test Strength: 83 %
- Line Coverage: 98 %
- Tests Executed: 577

```
root@DESKTOP-4K4RIIE:/mnt/c/sem7/st/ST_PROJECT# mvn clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< com.university:mutation-project >-------------------
[INFO] Building mutation-project 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ mutation-project ---
[INFO] Deleting /mnt/c/sem7/st/ST_PROJECT/target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ mutation-project ---
[INFO] skip non existing resourceDirectory /mnt/c/sem7/st/ST_PROJECT/src/main/resources
[INFO]
[INFO] --- compiler:3.8.1:compile (default-compile) @ mutation-project ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 9 source files to /mnt/c/sem7/st/ST_PROJECT/target/classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.819 s
[INFO] Finished at: 2025-11-25T15:02:51+05:30
[INFO] ------------------------------------------------------------------------
```

Figure 2: JUnit Test Execution Summary

# Pit Test Coverage Report

**Project Summary**

| Number of Classes | Line Coverage | | Mutation Coverage | | Test Strength | |
|---|---|---|---|---|---|---|
| 9 | 98% | 281/287 | 80% | 178/222 | 83% | 178/215 |

**Breakdown by Package**

| Name | Number of Classes | Line Coverage | | Mutation Coverage | | Test Strength | |
|---|---|---|---|---|---|---|---|
| com.university | 9 | 98% | 281/287 | 80% | 178/222 | 83% | 178/215 |

Report generated by PIT 1.15.0

Enhanced functionality available at arcmutate.com

Figure 3: PiTest tools screenshot

```
================================================================================
- Statistics
================================================================================
>> Line Coverage (for mutated classes only): 281/287 (98%)
>> Generated 222 mutations Killed 178 (80%)
>> Mutations with no coverage 7. Test strength 83%
>> Ran 577 tests (2.6 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  39.374 s
[INFO] Finished at: 2025-11-25T14:58:19+05:30
[INFO] ------------------------------------------------------------------------
```

Figure 4: Mutation Testing Summary

# IX Execution and Tooling

The project README provides detailed instructions for running the system, tests, and mutation testing workflow. These have been expanded here:

To execute the system from scratch:

1. Install Java 17 and Maven.
2. Compile the project using `mvn clean compile`.
3. Run the application via `java -cp target/classes com.university.Main`.
4. Execute all tests with `mvn test`.
5. Run mutation testing using `mvn org.pitest:pitest-maven:mutationCoverage`.
6. Open the report at: `target/pit-reports/index.html`.

The project uses:

- Java (JDK 17)
- Maven build system
- JUnit 5
- PITest Mutation Testing Framework
- Visual Studio Code

# X Code Flow Summary

This project simulates the core operations of a university system by establishing explicit data flow and inter–module dependencies among the eight principal classes in the `src/main/java/com/university` directory. The overall system behavior emerges from the interaction between initialization, enrollment, academic state transitions, library workflows, and the subsequent verification performed by mutation testing.

## X.1 Setup and Initialization

Execution begins in the `Main.java` class, which is responsible for constructing the foundational objects required by the system. These include the `Department` for budget management, `Faculty` assigned to courses, `Course` objects created with specified capacity and assigned instructors, `Student` objects that serve as active participants in the system, and finally the `EnrollmentService` which orchestrates interactions between students and courses. This initialization stage establishes all stateful components that the remainder of the system relies on.

## X.2 Enrollment and Integration Logic

The enrollment sequence is triggered when `Main` invokes the method:

```
EnrollmentService.enroll(Student, Course)
```

Within this method, the system enforces a series of integration-level constraints:

- It first checks whether the student is on academic probation using `isProbation()`.
- It verifies whether the targeted course has capacity for additional enrollment.
- It prints status messages summarizing the outcome of each check, enabling logging validation via later tests.
- On successful validation, the method updates the course's internal list of enrolled students.

This stage demonstrates multi-class interaction and contains multiple boolean guards that are intentionally mutation-sensitive.

## X.3 Academic State Update

After the enrollment flow, `Main` updates student academic progress via:

```
Student.updateAcademicRecord()
```

This method performs weighted GPA computation, accumulation of earned credits, and updates to academic standing. Because this method uses arithmetic accumulation and condition-sensitive GPA thresholds, it is heavily targeted by mutation operators involving mathematical replacement, condition negation, and assignment alteration. The student's academic state evolves based on historical performance, making this a crucial component of the system's correctness.

## X.4 Library Workflow and Utility Operations

The next stage of execution introduces the library subsystem, where `Main` initializes a `LibraryBook` and a `LibrarySystem`. When `returnBook()` is invoked, the system executes its overdue fine calculation logic, processes date boundaries including year wrap-around, updates the book's state, and validates borrower interactions. The logic in this module includes several arithmetic and boundary conditions that were deliberately included to support mutation testing.

## X.5 Test Verification Flow

The PITest mutation testing cycle validates all operational flows using a carefully constructed suite of tests. These tests include:

- **OutputCaptureTest**: Captures console output generated by `Main` and ensures expected messages, such as enrollment success notifications, are correctly printed. This test specifically kills void-method call mutants that remove `System.out.println()` statements.
- **ExceptionBoosterTest** and **BoundarySniperTest**: Rigorously test constructors and validation logic in `Person`, `Student`, and `Faculty`. These tests ensure that input constraints, boundary values, and arithmetic rules behave correctly and kill statement-deletion and conditional-boundary mutants.

Collectively, this verification flow ensures the correctness of enrollment logic, GPA computation, fine calculation, constructor validation, and console output behavior. The system's behavior is therefore validated across both functional and mutation-based perspectives, ensuring a robust and fault-resistant implementation.

# XI AI Usage Declaration

In accordance with course requirements, the project acknowledges the use of AI tools. Google Gemini was used for generating initial boilerplate structure for the classes, suggesting edge case tests, providing configuration snippets for mutation testing, and offering documentation support. All AI-generated content was manually validated, corrected, and extended by both contributors.

# XII Conclusion

The University Management System demonstrates a complete integration of software engineering principles, testing methodologies, and mutation analysis. The achieved Line coverage- 98%, Mutation Coverage- 80% and Test Strength of- 83% confirms that the test suite is sufficiently rigorous, effectively detecting subtle logical faults and guarding against regression risks. The project validates the critical educational insight that high line coverage alone is not a guarantee of correctness; instead, mutation testing serves as a more reliable indicator of test suite strength.

The system design, testing strategy, and tooling are scalable and can be extended with multithreading, concurrency tests, stress testing, and additional mutation operators.